
BÁO CÁO BÀI TẬP VỀ NHÀ 5

GIẢI HỆ PHƯƠNG TRÌNH TUYẾN TÍNH BẰNG PHƯƠNG PHÁP JACOBI VÀ GAUSS-SEIDEL

Họ và tên: Nguyễn Trần Gia Bảo

MSSV: 23137003

Môn học: Vật lý tính toán (PHY10532)

Giảng viên: TS. Vũ Quang Tuyên

June 30, 2026

Contents

1. Giới thiệu và nội dung bài tập

Bài tập yêu cầu giải hệ phương trình tuyến tính $Ax = B$ bằng hai phương pháp lặp: **Jacobi** và **Gauss–Seidel**. Cụ thể gồm hai phần:

1. **Bài 1:** Giải hệ 4×4 cho trước bằng cả hai phương pháp, so sánh tốc độ hội tụ.
2. **Bài 2 (Bài toán cột kéo):** Thiết lập và giải hệ 8×8 xuất phát từ bài toán cân bằng lực tại các nút của hệ thanh chịu lực.

2. Cơ sở lý thuyết

2.1. Bài toán tổng quát

Xét hệ n phương trình tuyến tính với n ẩn:

$$Ax = B, \quad (1)$$

trong đó $A \in \mathbb{R}^{n \times n}$ là ma trận hệ số, $x \in \mathbb{R}^n$ là vector ẩn và $B \in \mathbb{R}^n$ là vector vế phải.

Để giải bằng phương pháp lặp, ta tách ma trận A thành

$$A = D + L + U,$$

với D là ma trận đường chéo, L là phần tam giác dưới (không chứa đường chéo) và U là phần tam giác trên. Điều kiện tiên quyết là mọi phần tử đường chéo phải khác không: $a_{ii} \neq 0$ với mọi i .

2.2. Phương pháp Jacobi

Từ phương trình thứ i của hệ Phương trình (1):

$$\sum_{j=1}^n a_{ij} x_j = b_i,$$

ta tách riêng x_i :

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j \right). \quad (2)$$

Trong phương pháp Jacobi, ở mỗi bước lặp thứ $(k+1)$, ta dùng *toàn bộ* giá trị cũ $x^{(k)}$ để tính giá trị mới:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n. \quad (3)$$

Dưới dạng ma trận, công thức lặp Jacobi có thể viết:

$$x^{(k+1)} = D^{-1}(B - (L + U)x^{(k)}) = -D^{-1}(L + U)x^{(k)} + D^{-1}B.$$

Ma trận lặp tương ứng là $T_J = -D^{-1}(L + U)$.

2.3. Phương pháp Gauss–Seidel

Ý tưởng cải tiến: khi tính $x_i^{(k+1)}$, các thành phần $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ đã được tính xong ở bước lặp hiện tại, nên ta dùng chúng thay vì giữ giá trị cũ:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n. \quad (4)$$

Dưới dạng ma trận:

$$x^{(k+1)} = -(D + L)^{-1} U x^{(k)} + (D + L)^{-1} B.$$

Ma trận lặp tương ứng là $T_{GS} = -(D + L)^{-1} U$.

2.4. Điều kiện hội tụ

Cả hai phương pháp hội tụ khi bán kính phổ (spectral radius) của ma trận lặp nhỏ hơn 1. Một điều kiện đủ phổ biến là **ma trận chéo trội nghiêm ngặt theo hàng**:

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|, \quad \forall i = 1, \dots, n. \quad (5)$$

Khi Phương trình (5) được thỏa, cả Jacobi lẫn Gauss–Seidel đều hội tụ. Trong thực tế, Gauss–Seidel thường hội tụ nhanh hơn Jacobi vì sử dụng thông tin mới nhất ngay khi có.

2.5. Tiêu chuẩn dừng

Quá trình lặp dừng khi:

$$\text{err}^{(k)} = \max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon, \quad (6)$$

với ε là ngưỡng sai số cho trước. Ngoài ra, ta giới hạn số lần lặp tối đa $N_{\max}$ để tránh vòng lặp vô hạn khi phương pháp không hội tụ.

2.6. Kỹ thuật phương trình chuẩn tắc (Normal Equations)

Trong bài toán cột kéo, ma trận A không chéo trội và có nhiều phần tử đường chéo bằng 0, nên Jacobi/Gauss–Seidel đều *không hội tụ* khi áp dụng trực tiếp. Để khắc phục, ta nhân cả hai vế của Phương trình (1) với A^T :

$$A^T A x = A^T B. \quad (7)$$

2.6.1. Chứng minh

1. **Tương đương nghiệm:** Nếu x^* là nghiệm của $Ax = B$ thì hiển nhiên $A^T A x^* = A^T B$. Ngược lại, nếu A khả nghịch ($\det A \neq 0$) thì $A^T A$ cũng khả nghịch (vì $\det(A^T A) = (\det A)^2 \neq 0$), do đó hệ Phương trình (7) có nghiệm duy nhất trùng với nghiệm hệ gốc.

2. Ma trận $A^T A$ đối xứng:

$$M = A^T A \implies M^T = (A^T A)^T = A^T (A^T)^T = A^T A = M.$$

3. Ma trận $A^T A$ xác định dương: Với mọi $v \neq 0$:

$$v^T M v = v^T (A^T A) v = (Av)^T (Av) = \|Av\|^2.$$

Nếu A khả nghịch thì $Av \neq 0$ khi $v \neq 0$, do đó $\|Av\|^2 > 0$, tức M xác định dương.

4. Đường chéo luôn dương: Phần tử đường chéo của $M = A^T A$:

$$m_{ii} = \sum_{k=1}^n a_{ki}^2 > 0,$$

vì cột thứ i của A có ít nhất một phần tử khác 0. Do đó *không có đường chéo nào bằng 0*, giải quyết hoàn toàn vấn đề chia cho 0.

5. Gauss–Seidel luôn hội tụ: Theo định lý Ostrowski–Reich, phương pháp Gauss–Seidel hội tụ cho mọi ma trận đối xứng xác định dương. Vì $A^T A$ thỏa mãn cả hai tính chất này, Gauss–Seidel được đảm bảo hội tụ.

2.6.2. Tóm tắt ưu và nhược điểm

Ưu điểm:

- Đảm bảo đường chéo dương (không chia cho 0).
- Ma trận hệ số mới đối xứng, xác định dương.
- Gauss–Seidel được đảm bảo hội tụ theo lý thuyết.

Nhược điểm: Số điều kiện tăng bình phương:

$$\kappa(A^T A) = \kappa(A)^2,$$

nên tốc độ hội tụ có thể chậm hơn so với các phương pháp trực tiếp.

3. Phân tích chương trình

Chương trình được viết bằng Python trong notebook BTVN5.ipynb.

3.1. Hàm Jacobi

```
1 def Jacobi(N_max, epsilon, A, B):
2     n = len(B)
3     x_old = n * [0]
4     x_new = n * [0]
5     err_mang = n * [100]
6     err = np.max(err_mang)
```

```

7     x_matrix = []
8     err_matrix = []
9     x_matrix.append(x_old[:])
10    for q in range(N_max):
11        if err > epsilon:
12            for i in range(n):
13                tong = 0
14                for j in range(n):
15                    if i != j:
16                        tong = tong + A[i][j] * x_old[j]
17                x_new[i] = 1 / A[i][i] * (-tong + B[i])
18                err_mang[i] = np.abs(x_new[i] - x_old[i])
19            err = np.max(err_mang)
20            x_matrix.append(x_new[:])
21            err_matrix.append(err)
22            x_old[:] = x_new[:]
23        else:
24            break
25    return q, x_matrix, err_matrix

```

Giải thích chi tiết:

- **Dòng 2–5:** Khởi tạo vector nghiệm cũ và mới bằng 0; mảng sai số ban đầu bằng 100 để đảm bảo vòng lặp chạy ít nhất một lần.
- **Dòng 10–18:** Vòng lặp chính. Với mỗi ẩn x_i , tính tổng $\sum_{j \neq i} a_{ij} x_j^{(k)}$ dùng hoàn toàn giá trị cũ x_old rồi cập nhật theo Phương trình (3). Đây là đặc trưng Jacobi: *toàn bộ* x_old giữ nguyên trong suốt vòng lặp trong.
- **Dòng 19–22:** Tính sai số max-norm, lưu lịch sử nghiệm và sai số.
- **Dòng 23:** Cập nhật $x_old \leftarrow x_new$ cho bước tiếp.

3.2. Hàm Gauss–Seidel

```

1  def Gauss_Seidel(N_max, epsilon, A, B):
2      n = len(B)
3      x_old = n * [0]
4      x_new = n * [0]
5      err_mang = n * [100]
6      err = np.max(err_mang)
7      x_matrix = []
8      err_matrix = []
9      x_matrix.append(x_old[:])
10     for q in range(N_max):
11         if err > epsilon:
12             for i in range(n):
13                 tong1 = tong2 = 0
14                 for j in range(i):
15                     tong1 = tong1 + A[i][j] * x_new[j]
16                 for j in range(i+1, n):

```

```

17         tong2 = tong2 + A[i][j] * x_old[j]
18         x_new[i] = 1/A[i][i] * (-tong1 - tong2 + B[i])
19         err_mang[i] = np.abs(x_new[i] - x_old[i])
20         err = np.max(err_mang[:])
21         x_matrix.append(x_new[:])
22         err_matrix.append(err)
23         x_old[:] = x_new[:]
24     else:
25         break
26     return q, x_matrix, err_matrix

```

Điểm khác biệt quan trọng so với Jacobi nằm ở dòng 13–18:

- **tong1** (dòng 14–15): Tính $\sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)}$ – dùng giá trị *mới* $x_new[j]$ cho các ẩn $j < i$ đã cập nhật trong bước lặp hiện tại.
- **tong2** (dòng 16–17): Tính $\sum_{j=i+1}^n a_{ij} x_j^{(k)}$ – vẫn dùng giá trị *cũ* $x_old[j]$ cho các ẩn $j > i$ chưa cập nhật.
- Chính nhờ việc tận dụng ngay giá trị mới mà Gauss–Seidel thường hội tụ nhanh hơn Jacobi.

3.3. Hàm ghi file kết quả

```

1 def luu_file(q, x_matrix, err_matrix, filename):
2     so_nghiem = len(x_matrix[0])
3     heading = ""
4     for i in range(so_nghiem):
5         label = f"x_{i+1}"
6         heading = heading + f"{label:>15s} "
7     with open(f"{filename}", "w", encoding="utf-8") as file:
8         file.write(f"Ket qua tinh toan sau {q} lan lap:\n\n")
9         file.write(f"#" * 100 + "\n\n")
10        file.write(f"{'i':>5s}" + heading + f"{'sai_so':>20s}\n")
11        for i in range(q):
12            hang = f"{i+1:>5d}"
13            for x in x_matrix[i]:
14                hang = hang + f"{x:>15.9f} "
15            hang += f"{err_matrix[i]:>20.6e}\n"
16            file.write(hang)

```

Hàm này ghi toàn bộ lịch sử lặp (nghiệm ở mỗi bước và sai số tương ứng) ra file `.txt`, giúp theo dõi chi tiết quá trình hội tụ.

3.4. Hàm tính toán tổng hợp

```

1 def tinhtoan(N_max, epsilon, A, B, filename):
2     q_jacobi, x_matrix_jacobi, err_matrix_jacobi = \
3         Jacobi(N_max, epsilon, A, B)
4     q_gauss_seidel, x_matrix_gauss_seidel, err_matrix_gauss_seidel = \
5         Gauss_Seidel(N_max, epsilon, A, B)
6     filename_jacobi = f"{filename}_jacobi.txt"
7     filename_gauss_seidel = f"{filename}_gauss_seidel.txt"
8     luu_file(q_jacobi, x_matrix_jacobi,
9             err_matrix_jacobi, filename_jacobi)
10    luu_file(q_gauss_seidel, x_matrix_gauss_seidel,
11            err_matrix_gauss_seidel, filename_gauss_seidel)

```

Hàm tinhtoan đóng vai trò điều phối: gọi lần lượt Jacobi và Gauss–Seidel trên cùng hệ phương trình, rồi lưu kết quả vào hai file riêng biệt để tiện so sánh.

4. Kết quả và thảo luận

4.1. Bài 1: Hệ phương trình 4×4

4.1.1. Đề bài

Giải hệ phương trình:

$$\begin{bmatrix} 10 & -1 & 2 & 0 \\ -1 & 11 & -1 & 3 \\ 2 & -1 & 10 & -1 \\ 0 & 3 & -1 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 6 \\ 25 \\ -11 \\ 15 \end{bmatrix}, \quad (8)$$

với $\varepsilon = 10^{-9}$ và $N_{\max} = 1000$.

```

1 N_max = 1000
2 A = [[10, -1, 2, 0],
3      [-1, 11, -1, 3],
4      [2, -1, 10, -1],
5      [0, 3, -1, 8]]
6 B = [6, 25, -11, 15]
7 epsilon = 1e-9
8 tinhtoan(N_max, epsilon, A, B, "baitap1")

```

4.1.2. Kiểm tra điều kiện hội tụ

Xét tính chéo trội theo hàng:

$$\begin{aligned} |a_{11}| &= 10 > |-1| + |2| + |0| = 3, \\ |a_{22}| &= 11 > |-1| + |-1| + |3| = 5, \\ |a_{33}| &= 10 > |2| + |-1| + |-1| = 4, \\ |a_{44}| &= 8 > |0| + |3| + |-1| = 4. \end{aligned}$$

Mà trận A là chéo trội nghiêm ngặt $\Rightarrow$ cả Jacobi lẫn Gauss–Seidel đều đảm bảo hội tụ.